

Reading Material 1:

Operators in C — A Crash Course

(Learn by reading code: categories, precedence, and associativity)

1. The Big Picture: Categories

C operators are grouped into categories:

- **Arithmetic:** + - * / %
 - **Relational:** < <= > >= == !=
 - **Logical:** && || !
 - **Bitwise:** ~ & | ^ << >>
 - **Assignment (and compound assignment):** = += -= *= /= %= &= |= ^= <<= >>=
 - **Ternary:** ?:
 - **sizeof** (compile-time)
 - **Comma operator** , (sequence of expressions)
-

2. Read the Code — Arithmetic, Relational, Logical

```
int a = 10, b = 3;
printf("%d\n", a / b);    // 3 (integer division)
printf("%d\n", a % b);    // 1 (remainder)
printf("%d\n", a + b * 2); // 16 (* before +)
printf("%d\n", (a + b) * 2); // 26 (parentheses override precedence)

int x = 5, y = 7;
printf("%d\n", x < y && y < 20); // 1 (true): both conditions true
printf("%d\n", x == 5 || y == 0); // 1 (true): at least one true
printf("%d\n", !0);           // 1 (logical NOT)
```

Takeaways:

- * and / bind tighter than + and -.
- Comparisons happen *after* arithmetic.
- Logical && and || are evaluated after relational/equality.
- Parentheses make intent explicit.
- Logical operators short-circuit: if the result is already decided, the right side is skipped.

3. Assignment & Associativity

```
int a, b, c;
a = b = c = 1; // assignment is right-associative
// Evaluates as: a = (b = (c = 1));

int v = 10;
v += 3 * 2; // same as v = v + (3*2); now v = 16

int m = 8;
m <<= 1; // 16, left shift by 1 bit
```

- **Associativity** decides how operators of the *same precedence* are grouped.
- Assignments are **right-to-left**.
- Arithmetic like $a - b - c$ is **left-to-right** $\rightarrow (a - b) - c$.

4. Bitwise vs Logical

```
unsigned x = 0b0101; // 5
unsigned y = 0b0011; // 3

printf("%u\n", x & y); // 1 (bitwise AND)
printf("%u\n", x | y); // 7 (bitwise OR)
printf("%u\n", x ^ y); // 6 (bitwise XOR)
printf("%u\n", x << 1); // 10 (left shift)

// Logical vs bitwise:
printf("%d\n", (x && y)); // 1 (true)
printf("%d\n", (x & y)); // 1 here, but via bitwise calculation
```

Rule:

- Use `&& || !` for boolean logic.
- Use `& | ^ << >> ~` for bit manipulation.
- They look similar, but behave differently and have different precedence.

5. Precedence & Associativity — Debug by Parentheses

```
int a = 2, b = 3, c = 4;
int r1 = a + b * c;    // 14, '*' before '+'
int r2 = (a + b) * c;  // 20, parentheses first
int r3 = a < b == b < c; // (a<b) == (b<c) => 1==1 -> 1

int d = 1, e = 2, f = 3;
int r4 = d - e - f;    // (d - e) - f = -4 (left-associative)

int p = 1, *q, arr[3]={10,20,30};
q = arr;               // pointer to first element
int t1 = *q++;         // parsed as *(q++): t1=10, q points to 20
int t2 = (*q)++;       // increments value at q: arr[1]=21
```

Heuristic order:

- Postfix ($x++$) $\rightarrow$ Unary ($!x$, $*p$) $\rightarrow$ Multiplicative $\rightarrow$ Additive $\rightarrow$ Shifts $\rightarrow$ Relational $\rightarrow$ Equality $\rightarrow$ Bitwise $\rightarrow$ Logical $\rightarrow$ Ternary $\rightarrow$ Assignment $\rightarrow$ Comma.
- When unsure, **add parentheses**.

6. Ternary Operator $?:$

```
int max = (a > b) ? a : b;

const char* kind = (n % 2 == 0) ? "even" : "odd";
printf("%s\n", kind);
```

- Precedence: lower than logical operators, higher than assignment.
- **Right-associative:** $a ? b : c ? d : e$ parses as $a ? b : (c ? d : e)$.

7. `sizeof` & Comma

```
int x = 0;
printf("%zu\n", sizeof x); // OK without parentheses for variable
printf("%zu\n", sizeof(int)); // type requires parentheses
```

```
for (int i = 0, j = 10; i < j; i++, j--) {  
    printf("%d %d\n", i, j);  
}
```

- `sizeof` is compile-time for types and most expressions; it doesn't evaluate its operand.
- The **comma operator** evaluates left, then right, and yields the value of the right expression (commonly used in `for` loops).

8. Mini Precedence Chart (High → Low)

1. Postfix: `x++ x-- () [] . ->`
2. Unary: `++x --x + - ! ~ * & sizeof`
3. Multiplicative: `* / %`
4. Additive: `+ -`
5. Shifts: `<< >>`
6. Relational: `< <= > >=`
7. Equality: `== !=`
8. Bitwise: `& ^ |`
9. Logical: `&& ||`
10. Ternary: `?:` (right-assoc)
11. Assignment: `= += -= ...` (right-assoc)
12. Comma: `,`

Pro Tips

- Prefer **parentheses** over memorizing the full precedence table.
- Never rely on order of evaluation of sub-expressions — it can surprise you.
- Use logical operators for boolean logic and bitwise for masking.

Caution

Avoid expressions that modify a variable more than once without a sequence point, e.g.:

```
i = i++ + ++i; // ❌ Undefined behavior
```